

Scholarly Reference Validation System

Complete Codex Project Context / Engineering Handoff

Project type: Research automation / bibliographic reference validation

Primary language: Python

Current environment: Windows / IIT Delhi institutional network

Primary data source: Scopus CSV export

Initial dataset: 864 scholarly documents

Goal: Automatically validate references cited by these documents, including DOI validity, link accessibility, bibliographic correctness, and detection of valid-but-wrong DOIs.

1. PROJECT OBJECTIVE

Build a robust automated system that takes a Scopus export containing hundreds of scholarly documents and validates every reference contained in those documents.

The system must NOT treat reference validation as simply:

"Does the DOI URL open?"

Instead, it must answer several independent questions:

1. Does the reference contain a DOI?
2. If a DOI is present, does the DOI actually exist?
3. Does the DOI resolve?
4. Does the DOI resolve to a reachable webpage?
5. Does the publication represented by the DOI match the bibliographic information in the reference?
6. If the DOI is missing, can the intended DOI be recovered?
7. If a DOI exists but points to another publication, can this be detected?

8. Is the URL broken, redirected, inaccessible, or merely protected by authentication/bot blocking?
9. Where possible, is the cited publication retracted/corrected?
10. How confident is the automated decision?
11. Which cases require human review?

The system should ultimately produce a reference-level dataset suitable for research analysis.

2. IMPORTANT CONCEPTUAL DISTINCTION

A DOI being valid does NOT mean the reference is correct.

Example:

Reference:

```
Smith J. (2019).  
Machine Learning in Healthcare.  
Journal X.  
DOI: 10.1234/ABC
```

The DOI resolves successfully, but the DOI metadata says:

```
Smith J. (2020).  
Deep Learning in Finance.  
Journal Y.
```

Then:

```
DOI_EXISTS = TRUE  
DOI_RESOLVES = TRUE  
URL_WORKING = TRUE  
REFERENCE_MATCH = FALSE
```

Final classification should therefore be something like:

VALID_DOI_WRONG_REFERENCE

This distinction is a core requirement of the project.

3. CURRENT INPUT DATA

The user supplied the following Scopus CSV:

```
scopus_export_Sep 14-2026_f4d528d1-2e99-4a2a-b172-10d5d2688e60.csv
```

Current mounted path in the working environment:

```
/mnt/data/scopus_export_Sep 14-2026_f4d528d1-2e99-4a2a-b172-10d5d2688e60.csv
```

The file contains 864 rows/documents.

Columns:

```
Authors
Author full names
Author(s) ID
Title
Year
Source title
DOI
Link
References
EID
```

Important fields:

```
Title
Year
Source title
DOI
Link
References
EID
```

4. DATASET DIAGNOSTICS ALREADY PERFORMED

The CSV contains:

```
864 documents
```

Missing values:

Title	0
DOI	19
References	22
EID	0

The References column contains bibliographic references as a long text field.

Example:

```
Aithal PS; Aithal S., Patent analysis as a new scholarly research method, IJCSBE, pp. 33-47, (2018); Zameer M; Shimray SR., Patent analysis in India: A bibliometric study, ALIS, 72, 2, pp. 162-173, (2025); ...
```

IMPORTANT:

The semicolon character is used both:

1. between authors inside a single reference
2. between separate references

Therefore:

```
references.split(";")
```

IS WRONG.

A previous naive split produced approximately:

38,586 apparent reference fragments

but this is NOT a trustworthy reference count because author separators were incorrectly treated as reference boundaries.

Do NOT use the naive split dataset as ground truth.

5. ACTUAL REFERENCE FORMAT OBSERVED

References generally have structures such as:

```
Author A.; Author B.; Author C., Title, Journal, volume, issue, pp. 1-10, (2020);
```

or:

```
Author A; Author B., Title, Journal, 10, 2, pp. 100-110, (2019);
```

or incomplete records such as:

```
Salman S.H., (2017);
```

or:

```
Corona virus (COVID-19), dashboard;
```

or:

```
WHO R&D blueprint ovel coronavirus Covid-19 animal models, (2020);
```

Some records contain no DOI.

Some records contain no journal.

Some records contain incomplete metadata.

Some records have unusual punctuation/abbreviations.

Therefore the parser must be tolerant and should preserve the original raw reference.

6. NON-NEGOTIABLE DATA PRINCIPLE

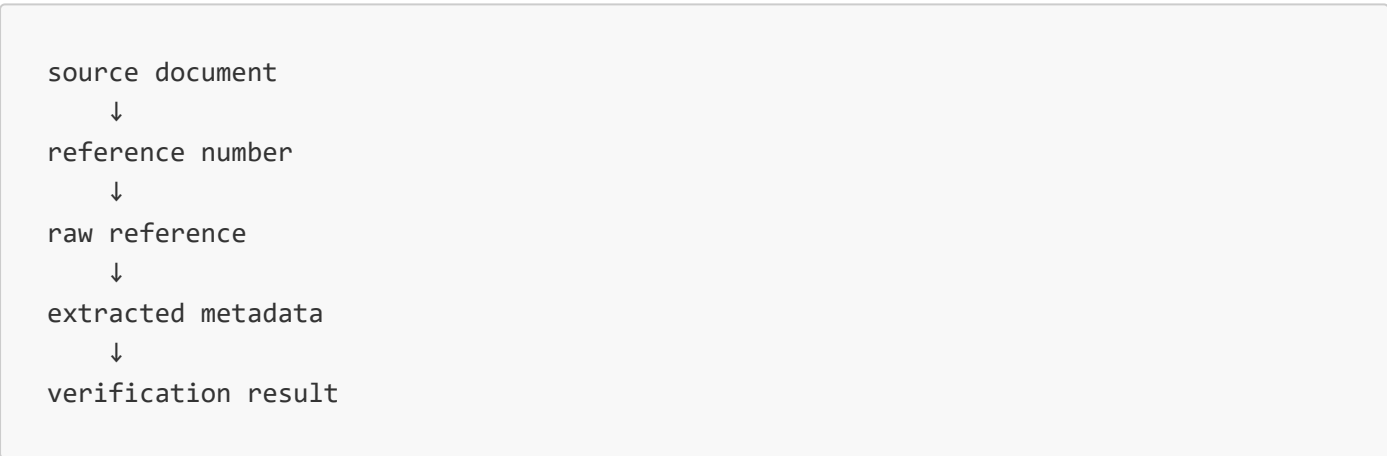
NEVER discard the original reference string.

Every parsed reference must contain:

```
raw_reference
```

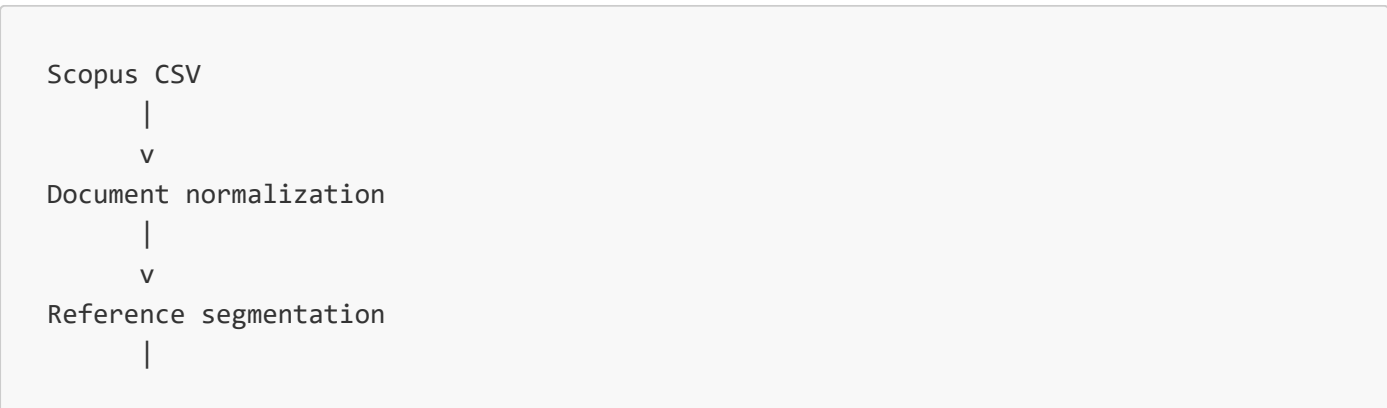
This is required for auditing automated decisions.

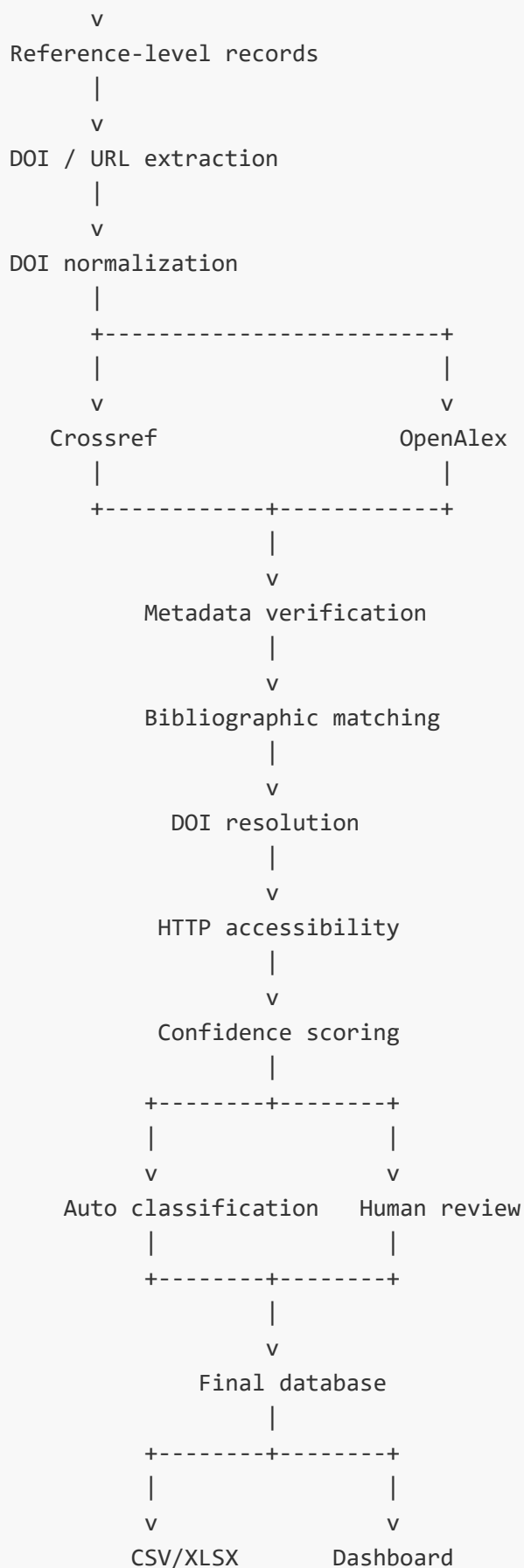
Every downstream result must be traceable to:



7. RECOMMENDED ARCHITECTURE

Overall pipeline:





8. RECOMMENDED TECHNOLOGY STACK

Primary:

Python 3.11+

Data processing:

pandas

HTTP:

requests
httpx

Parsing:

re
BeautifulSoup
lxml

Bibliographic matching:

rapidfuzz

Potential later semantic matching:

sentence-transformers

PDF extraction if required:

PyMuPDF
GROBID

Scopus browser automation only if necessary:

Playwright

Database:

```
SQLite initially
```

Potential production database:

```
PostgreSQL
```

Output:

```
pandas  
openpyxl
```

Optional dashboard:

```
Streamlit  
or Power BI
```

9. SCOPUS API STRATEGY

The user has institutional access to Scopus through IIT Delhi and is working from the IIT Delhi network.

The preferred approach is:

```
OFFICIAL ELSEVIER / SCOPUS API
```

NOT:

```
scraping Scopus UI as the primary architecture.
```

Scopus APIs can provide document metadata, and Elsevier provides reference retrieval functionality through appropriate API resources/entitlements.

The user may have to obtain an Elsevier API key through:

```
https://dev.elsevier.com/
```

DO NOT put the API key in source code.

Use:

```
ELSEVIER_API_KEY
```

environment variable.

Example:

```
X-ELS-APIKey
```

HTTP header.

Institutional access/subscription/API entitlement must be respected.

The user's IITD Wi-Fi may allow IP-based institutional authentication, but API entitlement still needs to be tested.

10. SCOPUS API TEST

Before building the full API layer, test:

```
GET  
https://api.elsevier.com/content/search/scopus
```

Example:

```
query = TITLE-ABS-KEY(machine learning)
```

Expected:

```
HTTP 200
```

and JSON containing Scopus search records.

Important response headers may include rate-limit information such as:

```
X-RateLimit-Limit  
X-RateLimit-Remaining
```

The production system must respect API quotas.

11. SCOPUS SEARCH URL

The user's Scopus UI search URL is:

```
https://www.scopus.com/pages/search/publications?searchId=718b7367-fd8b-4990-b445-5c75ffd6ecee
```

This corresponds to approximately 800+ documents.

However:

```
searchId
```

is a web UI search-session identifier.

Do NOT assume it can directly be passed to the API.

The actual Scopus query needs to be identified/reconstructed if the API is used to reproduce the search.

Fortunately, the user has already exported the results, so the current project can start from the CSV without reproducing the Scopus search.

12. IMPORTANT DISCOVERY FROM CURRENT CSV

The Scopus CSV ALREADY contains a:

References

column.

Therefore:

We do NOT need to scrape the Scopus UI merely to obtain the reference list.

The CSV can be used as the initial input.

This makes the project:

```
graph TD; CSV --> reference_parser[reference parser]; reference_parser --> verification;
```

rather than:

```
graph TD; Scopus_UI_scraper[Scopus UI scraper] --> references;
```

Browser automation should only be added if a later stage genuinely requires it.

13. PHASE 1 — REFERENCE SEGMENTATION

Current phase.

Goal:

Convert:

```
one Scopus document row  
+  
long References string
```

into:

```
one row per actual reference.
```

Output structure:

```
source_eid  
source_title  
source_year  
source_doi  
reference_no  
raw_reference
```

Additional fields may include:

```
reference_year_detected  
has_doi  
has_url  
needs_review
```

DO NOT claim that segmentation is perfect.

Because semicolons are overloaded, use bibliographic structure to determine reference boundaries.

14. REFERENCE BOUNDARY HEURISTIC

The observed format strongly suggests that publication years are useful boundaries.

Many references terminate in:

```
(2020)
(2019)
(2018)
etc.
```

A semicolon followed by a new author/name after a completed year is therefore a strong reference boundary.

However, some references:

```
have no year
have incomplete metadata
contain multiple years in titles
contain books/conference proceedings
have malformed records
```

Therefore:

```
year-based parsing = heuristic
```

NOT:

```
guaranteed truth.
```

Every uncertain record should be marked:

```
needs_review = TRUE
```

15. PARSER DESIGN PRINCIPLE

The parser should:

1. Normalize whitespace.
2. Iterate through semicolon-separated tokens.

3. Maintain a current reference buffer.
4. Treat a segment as a likely reference ending when it contains a publication-year marker such as:

(2020)
5. Emit the accumulated reference.
6. Preserve incomplete trailing material.
7. Flag suspicious/no-year references.
8. Never silently discard text.

Potential regex:

```
r"\\(\\s*(?:19|20)\\d{2}[a-z]?\\s*)"
```

This is only one structural signal.

The parser should eventually be improved by testing against manually verified references.

16. PHASE 1 OUTPUT

Recommended:

```
phase1_reference_level.csv
```

Columns:

```
source_eid
source_title
source_year
source_doi
reference_no
raw_reference
reference_year_detected
extracted_doi
```

```
has_doi
has_url
has_year
year_marker_count
needs_review
```

Also create:

```
phase1_manual_review_sample.csv
```

containing suspicious parser outputs.

17. PHASE 2 — DOI EXTRACTION

For each raw reference, attempt to detect:

```
DOI
```

Possible formats:

```
doi:10.xxxx/xxxxx
```

```
DOI: 10.xxxx/xxxxx
```

```
https://doi.org/10.xxxx/xxxxx
```

```
https://dx.doi.org/10.xxxx/xxxxx
```

Normalize to:

```
10.xxxx/xxxxx
```

Do not include:

```
https://doi.org/
```

in the canonical DOI field.

Strip trailing punctuation such as:

```
.  
,  
;  
)
```

but be careful not to corrupt legitimate DOI characters.

18. DOI EXTRACTION EXPECTATION

The current CSV sample contains very few explicit DOI strings in the References column.

This is important.

Therefore:

```
DOI extraction
```

will not solve the entire problem.

Many references will have:

```
DOI_MISSING
```

and the system must perform bibliographic lookup to recover a likely DOI.

19. PHASE 3 — DOI EXISTENCE / RESOLUTION

For references with a DOI:

```
normalized DOI  
|  
v
```

```
DOI resolver
  |
  v
https://doi.org/<DOI>
  |
  v
HTTP result
  |
  v
redirect chain
  |
  v
final URL
```

Record:

```
doi_exists
doi_resolves
final_url
http_status
redirect_count
response_time
checked_at
```

Potential statuses:

```
200
301
302
403
404
410
timeout
DNS failure
```

20. HTTP STATUS INTERPRETATION

Do NOT classify:

```
200 = valid
404 = invalid
```

too simplistically.

Examples:

```
200
    likely reachable

301 / 302
    redirect; generally acceptable if final destination is valid

403
    may mean access restriction or bot protection
    NOT necessarily a broken DOI

401
    authentication required

404
    likely broken DOI/URL

410
    resource gone

timeout
    unable to verify

DNS failure
    domain resolution failure
```

Therefore separate:

```
DOI validity
```

from:

```
URL accessibility.
```

21. PHASE 4 — CROSSREF

Use Crossref as a major bibliographic metadata source.

For a DOI:

```
DOI
↓
Crossref
↓
title
authors
journal/container
year
volume
issue
pages
publisher
etc.
```

Crossref REST API supports DOI metadata retrieval and bibliographic searching.

Crossref requests should:

```
use caching
identify the client with a meaningful User-Agent
optionally provide mailto
respect rate limits
```

Do not repeatedly request the same DOI.

22. CROSSREF CACHE

Implement a local cache.

Example:

```
cache/crossref/<hash>.json
```

or a database table:

```
crossref_cache
```

Fields:

```
doi
response_json
fetched_at
status_code
```

If the same DOI appears in 100 references:

```
make 1 Crossref request
```

not:

```
100 requests.
```

23. PHASE 5 — OPENALEX

Use OpenAlex as an independent metadata source.

For each DOI where practical:

```
DOI
↓
OpenAlex
↓
work metadata
```

Use it as a second source for:

```
title
authors
publication year
venue
identifiers
```

Purpose:

```
cross-validation
```

If Crossref and OpenAlex independently identify the same work, confidence increases.

If they disagree:

```
flag for investigation.
```

24. PHASE 6 — BIBLIOGRAPHIC PARSING

The raw reference needs to be converted into fields such as:

```
cited_title  
cited_authors  
cited_year  
cited_journal  
cited_volume  
cited_issue  
cited_pages
```

The parser does NOT have to be perfect.

It should output:

```
extracted metadata  
extraction confidence  
raw reference
```

Potential approaches:

1. Rule-based parser
2. GROBID if full references/PDFs are available
3. LLM-assisted parsing for difficult cases
4. Hybrid approach

Recommended initial approach:

```
deterministic/rule-based parsing
```

then use:

GROBID / LLM

for difficult cases.

25. PHASE 7 — METADATA MATCHING

Compare:

reference metadata

against:

DOI-resolved metadata.

Fields:

title
authors
journal
year
volume
issue
pages

Example:

Reference:

Ngai E.; Gunasekaran A.
A review for mobile commerce research and applications
Decision Support Systems
43, 1, pp. 3-15
(2007)

Resolved metadata:

```
Ngai, Eric W.T.  
Gunasekaran, Angappa  
A review for mobile commerce research and applications  
Decision Support Systems  
43  
1  
3-15  
2007
```

This should score very highly despite formatting differences.

26. TITLE MATCHING

Use normalized strings:

```
lowercase  
remove punctuation  
normalize whitespace  
normalize common Unicode variants  
optionally normalize common abbreviations
```

Then use:

```
RapidFuzz
```

Possible metrics:

```
ratio  
WRatio  
token_set_ratio  
token_sort_ratio
```

Title similarity is one of the strongest signals.

27. AUTHOR MATCHING

Author names are inconsistent.

Examples:

```
Smith J.
```

```
John Smith
```

```
Smith, John
```

```
J. Smith
```

Normalize names before comparison.

Potential strategy:

```
surname
```

```
first initial
```

```
first name where available
```

Compare author lists.

Do not require exact string equality.

28. JOURNAL MATCHING

Journal names may appear as:

```
Decision Support Sys
```

versus:

```
Decision Support Systems
```

Need normalization.

Potential future enhancement:

```
journal abbreviation dictionary
```

or metadata-based alias matching.

Journal matching should be a supporting signal, not the only decision.

29. YEAR MATCHING

Simple but useful.

Possible rules:

```
exact year → strong match
```

```
±1 year → possible match
```

```
>1 year difference → suspicious
```

But note:

```
online-first date
```

```
issue publication date
```

```
print publication date
```

can differ.

Therefore year mismatch should reduce confidence rather than automatically mark wrong.

30. VOLUME / ISSUE / PAGE MATCHING

Where available:

```
volume
```

```
issue
```

```
first page
```

```
page range
```

are strong supporting evidence.

Do not require them when the source/reference doesn't contain them.

31. REFERENCE MATCH SCORE

Initial conceptual scoring:

title similarity	35–40%
author similarity	20–25%
journal similarity	10–15%
year match	10%
volume	5–10%
pages	5–10%

These weights are NOT scientifically fixed.

They must be calibrated against a manually labelled sample.

Do not present arbitrary thresholds as validated research results.

32. HUMAN-LABELED CALIBRATION

Before running the full 864-document dataset:

Create a manually labelled sample.

For example:

200 references

Human labels:

CORRECT
WRONG_DOI
WRONG_METADATA
INVALID_DOI
AMBIGUOUS
DOI_MISSING
etc.

Then evaluate the automated scoring system.

Tune:

```
weights
thresholds
normalization rules
```

Measure:

```
precision
recall
F1
false-positive rate
false-negative rate
```

This is important if the system will be used for research publication.

33. FINAL STATUS TAXONOMY

Do NOT use only:

```
WORKING
BROKEN
```

Use more detailed statuses.

Recommended:

```
VALID_CORRECT

VALID_REDIRECT_CORRECT

VALID_DOI_WRONG_REFERENCE

INVALID_DOI

BROKEN_URL

ACCESS_RESTRICTED

DOI_MISSING
```

DOI_RECOVERED

DOI_RECOVERY_UNCERTAIN

METADATA_MISMATCH

RETRACTED_REFERENCE

AMBIGUOUS

UNVERIFIED

These statuses may evolve during development.

34. DOI-MISSING RECOVERY

For a reference with no DOI:

```
raw reference
  ↓
parse title/authors/year/journal
  ↓
Crossref bibliographic search
  ↓
OpenAlex search
  ↓
Scopus search if API available
  ↓
candidate works
  ↓
metadata similarity
  ↓
confidence
  ↓
DOI_RECOVERED or AMBIGUOUS
```

Example:

```
Reference:
Machine Learning Applications in Healthcare
Smith J.
2019
```

Candidate:

```
10.5678/XYZ
```

If:

```
title similarity = 0.98
author similarity = 1.00
journal = match
year = match
```

then:

```
DOI_RECOVERED
confidence = HIGH
```

35. VALID DOI BUT WRONG CITATION

This is a key output.

Example:

Reference:

```
Smith J.
Machine Learning Applications in Healthcare.
2019.
DOI: 10.1234/ABC
```

DOI resolves to:

```
Smith J.
Deep Learning Applications in Finance.
2020.
```

Result:

```
DOI_EXISTS = TRUE
DOI_RESOLVES = TRUE
```

```
URL_WORKING = TRUE
METADATA_MATCH = FALSE
FINAL_STATUS = VALID_DOI_WRONG_REFERENCE
```

This must not be classified as BROKEN.

36. RETRACTION / PUBLICATION STATUS

Potential future stage.

After correctly identifying the cited work:

```
DOI
  ↓
publication status
  ↓
retracted?
corrected?
updated?
```

If the paper is correctly cited but retracted:

```
REFERENCE_MATCH = CORRECT
PUBLICATION_STATUS = RETRACTED
```

This should be separate from reference correctness.

Do not automatically call a retracted reference "invalid."

37. DATABASE DESIGN

Use three logical tables initially.

documents

```
document_id
eid
scopus_id
doi
title
authors
journal
year
source_url
```

references

```
reference_id
document_id
reference_number
raw_reference
cited_title
cited_authors
cited_year
cited_journal
cited_volume
cited_issue
cited_pages
extracted_doi
extracted_url
```

verification

```
reference_id
normalized_doi
doi_exists
doi_resolves
http_status
final_url
redirect_count
crossref_found
openalex_found
resolved_title
resolved_authors
resolved_journal
resolved_year
resolved_volume
resolved_issue
resolved_pages
```

```
title_similarity
author_similarity
journal_similarity
year_match
volume_match
pages_match
metadata_confidence
final_status
checked_at
```

38. RECOMMENDED PROJECT STRUCTURE

Create:

```
reference_validator/

  README.md

  requirements.txt

  .env.example

  config.py

  main.py

  data/
    input/
    intermediate/
    output/

  src/
    __init__.py

    io/
      csv_loader.py
      writers.py

    parsing/
      reference_splitter.py
      citation_parser.py
      doi_extractor.py
      normalizer.py

    providers/
      crossref.py
      openalex.py
```

```
scopus.py
doi_resolver.py

matching/
  title_matcher.py
  author_matcher.py
  metadata_matcher.py
  scoring.py

validation/
  url_checker.py
  doi_validator.py
  publication_status.py

pipeline/
  phase1.py
  phase2.py
  phase3.py
  phase4.py

review/
  review_export.py

models/
  document.py
  reference.py
  verification.py

cache/
  crossref/
  openalex/
  doi/

tests/
  test_reference_splitter.py
  test_doi_extractor.py
  test_normalization.py
  test_matching.py
  test_scoring.py
```

39. DEVELOPMENT PHILOSOPHY

Build incrementally.

DO NOT start with all 864 documents.

Recommended development:

Phase 1A

Use:

10 documents

Test:

reference segmentation

Phase 1B

Use:

50 documents

Measure parser quality.

Phase 2

Test:

DOI extraction

Phase 3

Test:

Crossref/OpenAlex

Phase 4

Test:

metadata matching

Phase 5

Test:

HTTP resolution

Phase 6

Combine everything.

Phase 7

Run:

100 documents

Phase 8

Run:

full 864 documents

40. CACHING REQUIREMENT

All external API calls should be cached.

Never repeatedly call:

```
Crossref  
OpenAlex  
Scopus
```

for the same identifier.

Cache key:

```
normalized DOI
```

or:

```
normalized bibliographic query
```

This makes reruns faster and reduces API load.

41. RETRY POLICY

External requests may fail temporarily.

Implement:

```
exponential backoff
```

for:

```
429  
500  
502
```

503
504

Do NOT aggressively retry:

400
401
403
404

unless there is a specific reason.

42. URL CHECKING SHOULD BE POLITE

For external URLs:

```
set timeout  
  
use identifiable User-Agent  
  
follow redirects  
  
limit concurrency  
  
cache result  
  
record timestamp
```

Do not create a huge uncontrolled request storm.

43. CONCURRENCY

Do not immediately launch hundreds/thousands of simultaneous HTTP requests.

Start conservatively.

Example:

5-10 concurrent requests

and tune based on provider limits.

Scopus API calls should be particularly respectful of quotas.

44. LOGGING

Every stage should log:

```
document ID
reference ID
provider
request
response status
error
retry
final decision
```

Use Python logging.

Recommended levels:

```
DEBUG
INFO
WARNING
ERROR
```

45. ERROR HANDLING

One bad reference must NOT stop the complete pipeline.

For example:

```
Reference 1 → success
Reference 2 → success
```

```
Reference 3 → Crossref timeout  
Reference 4 → success
```

Reference 3 becomes:

```
UNVERIFIED
```

and processing continues.

46. REPRODUCIBILITY

Every verification result should contain:

```
checked_at
```

and ideally:

```
provider  
provider_version/API endpoint  
request identifier if available
```

Because metadata and URLs can change over time.

A future rerun should be able to compare:

```
September 2026 verification
```

versus:

```
January 2027 verification
```

47. OUTPUT FILES

Recommended final outputs:

documents_normalized.csv

references_parsed.csv

doi_validation_results.csv

reference_validation_results.csv

human_review.csv

summary_statistics.csv

Potential Excel workbook:

reference_validation_results.xlsx

Sheets:

Summary

Documents

References

DOI Results

Suspicious

Human Review

Errors

48. SUMMARY DASHBOARD

Eventually show:

Documents analyzed

References analyzed

References with DOI

References without DOI

Valid DOIs

Invalid DOIs

Working URLs

Broken URLs

Access restricted

Correct references

Wrong DOI

DOI recovered

Ambiguous references
Retracted references

Example:

```
Documents: 864
References: 35,000+
DOI present: 9,000
DOI missing: 26,000
Valid DOI: 8,700
Wrong DOI: 180
Broken DOI: 120
DOI recovered: 4,500
Ambiguous: 600
```

Numbers above are examples only; never hard-code them.

49. MACHINE LEARNING — LATER, NOT FIRST

Do NOT begin with an ML model.

First build a deterministic baseline.

Features:

```
title_similarity
author_similarity
journal_similarity
year_difference
volume_match
page_match
crossref_match
openalex_match
doi_resolves
http_status
```

Then collect human labels.

Only after that consider:

logistic regression
gradient boosting
random forest
XGBoost
neural/embedding-based classifier

The model should predict:

CORRECT
WRONG
AMBIGUOUS

rather than replace the underlying evidence.

50. LLM USAGE

An LLM can be useful for ambiguous bibliographic cases.

Example:

Reference title:
Recent advances in deep neural nets

Resolved title:
Recent advances in deep neural networks: A review

The LLM may help determine whether the records probably refer to the same publication.

But:

LLM decision

must never be the sole evidence.

Always retain:

raw reference
extracted metadata

provider metadata
deterministic similarity metrics
LLM decision
final confidence

51. HUMAN REVIEW SYSTEM

Human review should focus on uncertainty.

Example:

```
HIGH CONFIDENCE
  → automatic

MEDIUM
  → optional review

LOW
  → human review

AMBIGUOUS
  → human review
```

The objective is:

```
automate 80-95%+
review only difficult cases
```

rather than attempting false 100% automation.

52. TEST CASES REQUIRED

Create unit tests for:

DOI

10.1234/example

doi:10.1234/example

https://doi.org/10.1234/example

https://dx.doi.org/10.1234/example.

Author normalization

Smith J.

Smith, John

John Smith

Titles

Machine Learning in Healthcare

Machine learning in health care

Year

(2020)

(2020a)

DOI mismatch

citation metadata != DOI metadata

Broken URL

HTTP 404

Restricted

HTTP 403

Redirect

HTTP 301 → 200

53. SECURITY

Never hard-code:

```
Elsevier API key  
credentials  
institutional login  
passwords
```

Use:

```
.env
```

and:

```
.gitignore
```

Example:

```
ELSEVIER_API_KEY=  
CROSSREF_MAILTO=  
OPENALEX_EMAIL=
```

Never commit `.env`.

54. IMPORTANT CURRENT STATUS

At the moment, the project has:

```
Original Scopus CSV  
  ↓  
Dataset inspection completed  
  ↓  
Naive parser identified as unsafe  
  ↓  
Correct parser design is the next implementation step
```

A preliminary naive output was generated during development, but it **MUST NOT** be considered authoritative because semicolon splitting incorrectly treats author separators as reference separators.

The next implementation should create:

```
phase1_reference_level_v2.csv
```

using a year-aware/reference-aware parser.

55. IMMEDIATE NEXT TASK

DO THIS NEXT:

1. Load the supplied Scopus CSV.
2. Implement a proper heuristic reference splitter.
3. Do NOT use simple `split(";")`.

4. Preserve every raw reference.
 5. Use publication-year boundaries as one structural signal.
 6. Flag: no-year references multiple-year references suspiciously long records
incomplete records
 7. Produce: phase1_reference_level_v2.csv phase1_manual_review_sample.csv
 8. Generate summary statistics: document count parsed reference count average
references/document min max no-year count suspicious count explicit DOI count
URL count
 9. Print 20 representative parsed references for inspection.
 10. Do NOT proceed to external APIs until the parser output has been validated.
-

56. AFTER PHASE 1 IS VALIDATED

Proceed in this exact order:

PHASE 2
DOI extraction + normalization

↓

PHASE 3
Crossref lookup/cache

↓

PHASE 4
OpenAlex lookup/cache

↓

PHASE 5
DOI resolver + HTTP checking

↓

PHASE 6
Bibliographic metadata matching

↓

PHASE 7
Missing DOI recovery

↓

PHASE 8

Confidence scoring

↓

PHASE 9

Human review export

↓

PHASE 10

Full 864-document run

↓

PHASE 11

Statistics/dashboard

↓

PHASE 12

Optional ML classifier

57. CORE PRINCIPLE

The system should answer:

"Is this reference correct and reachable?"

not merely:

"Does this DOI open?"

The final evidence chain should be:

SOURCE DOCUMENT

↓

RAW REFERENCE

↓

EXTRACTED BIBLIOGRAPHY

↓

DOI

↓

DOI EXISTENCE

↓

DOI RESOLUTION
↓
RESOLVED PUBLICATION
↓
METADATA COMPARISON
↓
URL ACCESSIBILITY
↓
PUBLICATION STATUS
↓
CONFIDENCE
↓
FINAL STATUS

Every automated decision must be auditable.

58. SUCCESS CRITERIA

The project is successful when:

1. All 864 Scopus documents can be processed without crashing.
 2. References are segmented with high accuracy.
 3. Every reference retains its raw text.
 4. DOIs are normalized consistently.
 5. DOI existence is checked independently.
 6. URL resolution is checked independently.
 7. Bibliographic metadata is compared.
 8. Valid-but-wrong DOIs are detected.
 9. Missing DOIs can be recovered when confidence is high.
 10. Access-restricted pages are not incorrectly labelled broken.
 11. Ambiguous cases are routed to human review.
 12. All API responses are cached.
 13. The complete pipeline is reproducible.
 14. Results can be exported to CSV/XLSX.
 15. Manual labels can eventually be used to improve the classifier.
-

59. IMPORTANT WARNING FOR CODEX

Do NOT:

- scrape Scopus unnecessarily
- assume `References.split(";")` is correct
- equate HTTP 200 with citation correctness
- equate HTTP 403 with a broken DOI
- equate DOI existence with reference correctness
- discard raw references
- make thousands of uncached API requests
- hard-code API credentials
- automatically label ambiguous cases as wrong
- start with ML before establishing a deterministic baseline
- claim accuracy without a manually labelled validation set

DO:

- build incrementally
- preserve raw data
- cache API results
- maintain provenance
- use multiple metadata providers
- separate DOI validity from bibliographic correctness
- separate URL accessibility from DOI validity
- quantify uncertainty
- maintain human-review capability

60. EXPECTED FIRST COMMAND

Codex should begin by inspecting the supplied CSV and implementing Phase 1.

Suggested starting command:

```
python -m src.pipeline.phase1
```

or initially:

```
python phase1_reference_parser.py
```

The first deliverable should be the corrected reference-level dataset, NOT the complete API system.

Once Phase 1 is validated, continue to DOI extraction and external metadata verification.

END OF PROJECT CONTEXT